

 المعهد العالي للإعلامية و المعلوماتية بقرطاج	Matière : Atelier C++ TP 2 – Syntaxe de base du C++	Année Scolaire 2025-2026 Enseignante : Amani FALLAH
---	--	--

Exercice 1 : allocation statique / allocation dynamique

Déclarer un pointeur vers un entier puis construire cet entier et l'initialiser avec une valeur entière. Afficher la valeur de l'entier et l'adresse de son emplacement mémoire.

Refaire la même chose avec un réel.

```
#include <iostream>
using namespace std;

int main() {
    // --- ① Pointeur vers un entier ---
    int a = 42;           // déclaration et initialisation de l'entier
    int* pInt = &a;       // déclaration du pointeur vers int et affectation de
                          // l'adresse de a

    cout << "Valeur de l'entier a : " << a << endl;
    cout << "Adresse mémoire de a : " << &a << endl;
    cout << "Contenu du pointeur pInt (adresse de a) : " << pInt << endl;
    cout << "Valeur pointée par pInt (*pInt) : " << *pInt << endl;

    cout << "-----" << endl;

    // --- ② Pointeur vers un réel ---
    float b = 3.14;       // déclaration et initialisation du réel
    float* pFloat = &b;   // pointeur vers float

    cout << "Valeur du réel b : " << b << endl;
    cout << "Adresse mémoire de b : " << &b << endl;
    cout << "Contenu du pointeur pFloat (adresse de b) : " << pFloat << endl;
    cout << "Valeur pointée par pFloat (*pFloat) : " << *pFloat << endl;

    return 0;
}
```

Exercice 2 : passage par valeur / passage par référence / passage par adresse

Ecrire trois version d'une fonction qui teste l'égalité entre deux entiers (utiliser le passage par valeur, par adresse et par référence). Tester cette fonction dans main.

```
#include <iostream>
using namespace std;

// Passage par valeur
bool egalValeur(int a, int b) {
    return a == b;
}

// Passage par adresse (pointeurs)
```

```

bool egalAdresse(int* a, int* b) {
    return *a == *b;
}

// Passage par référence
bool egalReference(int& a, int& b) {
    return a == b;
}

int main() {
    int x, y;

    cout << "Entrez deux entiers : ";
    cin >> x >> y;

    cout << "\n--- Test passage par valeur ---" << endl;
    cout << (egalValeur(x, y) ? "Egaux" : "Différents") << endl;

    cout << "\n--- Test passage par adresse ---" << endl;
    cout << (egalAdresse(&x, &y) ? "Egaux" : "Différents") << endl;

    cout << "\n--- Test passage par référence ---" << endl;
    cout << (egalReference(x, y) ? "Egaux" : "Différents") << endl;

    return 0;
}

```

Exercice 3 : références et pointeurs

Écrire une fonction nommée `incrémenter()` permettant d'incrémenter la valeur d'une variable passée en paramètre et une fonction nommée `permuter()` permettant d'échanger les contenus de 2 variables de type `int` fournies en argument :

- en transmettant l'adresse des variables concernées (seule méthode utilisable en C) ;
- en utilisant la transmission par référence.

Dans les deux cas, écrire un programme (main) qui teste les deux fonctions.

Version Avec Pointeur :

```

#include <iostream>
using namespace std;

// Fonction qui incrémente une variable via un pointeur
void incrémenter(int* x) {
    (*x)++; // on déréférence puis on incrémente la valeur
}

// Fonction qui permute deux variables via des pointeurs
void permuter(int* a, int* b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main() {
    int a, b;
    cout << "entrer un entier" << endl;
    cin >> a;
}

```

```

    cout << "entrer un entier" << endl;
    cin >> b;

    cout << "=== Version avec pointeurs ===" << endl;
    cout << "Avant incrementer : a = " << a << endl;
    incrementer(&a);
    cout << "Après incrementer : a = " << a << endl;

    cout << "Avant permuter : a = " << a << ", b = " << b << endl;
    permuter(&a, &b);
    cout << "Après permuter : a = " << a << ", b = " << b << endl;

    return 0;
}

```

Version Avec Référence :

```

#include <iostream>
using namespace std;

// Fonction qui incrémente une variable via une référence
void incrementer(int& x) {
    x++;
}
// Fonction qui permute deux variables via des références
void permuter(int& a, int& b) {
    int temp = a;
    a = b;
    b = temp;
}

int main() {
    int a = 5, b = 10;

    cout << "\n=== Version avec références ===" << endl;
    cout << "Avant incrementer : a = " << a << endl;
    incrementer(a);
    cout << "Après incrementer : a = " << a << endl;

    cout << "Avant permuter : a = " << a << ", b = " << b << endl;
    permuter(a, b);
    cout << "Après permuter : a = " << a << ", b = " << b << endl;

    return 0;
}

```

Exercice 4 : fonctions / passage par références

Ecrire un programme en C++ qui permet de simuler le fonctionnement d'une calculatrice :

```
***** CALCULATRICE *****
* 1. Addition                **
* 2. Multiplication          **
* 3. Soustraction            **
* 4. Division                **
* 5. Quitter                 **
*****
Operation?
```

1. L'utilisateur choisira l'opération à effectuer et saisira les opérandes. Le programme affichera le résultat de l'opération. 2. Le programme effacera la console et affichera le menu de nouveau tant que l'utilisateur n'a pas choisi de quitter.

Pour effacer la console on peut utiliser l'instruction `system("cls")`.

```
#include <iostream>
#include <cstdlib> // pour system("cls")
using namespace std;

// ==== Fonctions ====

// Addition par référence
void addition(double& a, double& b, double& resultat) {
    resultat = a + b;
}

// Soustraction par référence
void soustraction(double& a, double& b, double& resultat) {
    resultat = a - b;
}

// Multiplication par référence
void multiplication(double& a, double& b, double& resultat) {
    resultat = a * b;
}

// Division par référence (avec vérification)
void division(double& a, double& b, double& resultat) {
    if (b != 0)
        resultat = a / b;
    else {
        cout << "Erreur : division par zéro !" << endl;
        resultat = 0;
    }
}

// ==== Fonction principale ====
int main() {
    int choix;
    double a, b, resultat;

    do {
        system("cls"); // efface l'écran (Windows)
        cout << "***** CALCULATRICE *****" << endl;
```

```

cout << "* 1. Addition          *" << endl;
cout << "* 2. Multiplication      *" << endl;
cout << "* 3. Soustraction         *" << endl;
cout << "* 4. Division              *" << endl;
cout << "* 5. Quitter                *" << endl;
cout << "*****" << endl;
cout << "Operation ? ";
cin >> choix;

if (choix >= 1 && choix <= 4) {
    cout << "\nEntrer le premier nombre : ";
    cin >> a;
    cout << "Entrer le deuxieme nombre : ";
    cin >> b;

    switch (choix) {
        case 1:
            addition(a, b, resultat);
            cout << "Resultat = " << resultat << endl;
            break;
        case 2:
            multiplication(a, b, resultat);
            cout << "Resultat = " << resultat << endl;
            break;
        case 3:
            soustraction(a, b, resultat);
            cout << "Resultat = " << resultat << endl;
            break;
        case 4:
            division(a, b, resultat);
            cout << "Resultat = " << resultat << endl;
            break;
    }

}

else if (choix != 5) {
    cout << "\nChoix invalide !" << endl;
}

if (choix != 5) {
    cout << "\nAppuyez sur une touche pour continuer...";
    cin.ignore();
    cin.get(); // attend une entrée pour revenir au menu
}

} while (choix != 5);

cout << "\nMerci d'avoir utilisé la calculatrice !" << endl;
return 0;
}

```

Exercice 5 : les nombres aléatoires

Ecrire une fonction qui permet de générer des réels (resp. des entiers) aléatoires dans un intervalle donné. Dans la fonction main faire l'appel 5 fois à cette fonction et afficher les nombres aléatoires générés. Utiliser la bibliothèque « random ».

Exercice 6 : fonction récursive

Écrire un programme qui demande à l'utilisateur de taper un entier N et qui calcule u(N) défini

par :

$u(0)=3$;

$u(n+1)=3.u(n)+4$.

Correction :

```
#include <iostream>
using namespace std;

int main() {
    int N;
    cout << "Entrez un entier N : ";
    cin >> N;

    int u = 3; // u(0)

    for (int i = 1; i <= N; i++) {
        u = 3 * u + 4;
    }

    cout << "u(" << N << ") = " << u << endl;

    return 0;
}
```